

Rilevamento e mitigazione automatica di attacchi DDoS in ambiente SDN tramite Machine Learning

Corso: Network and Cloud Infrastructures - Prof. Giorgio Ventre - a.a. 2025/2026
Studenti: DA INSERIRE - Matricole: DA INSERIRE
Repository GitHub: DA INSERIRE

Il progetto realizza una rete SDN emulata in Mininet nella quale un controller Ryu monitora i flussi OpenFlow, classifica il traffico diretto a un server protetto mediante un modello Random Forest e installa automaticamente regole DROP selettive verso le sorgenti sospette. La vittima e' l'host h4 (10.0.0.4). La decisione di blocco e' affidata al modello di Machine Learning e non a una soglia fissa.

Topologia core/access: 5 switch, 12 host, vittima h4

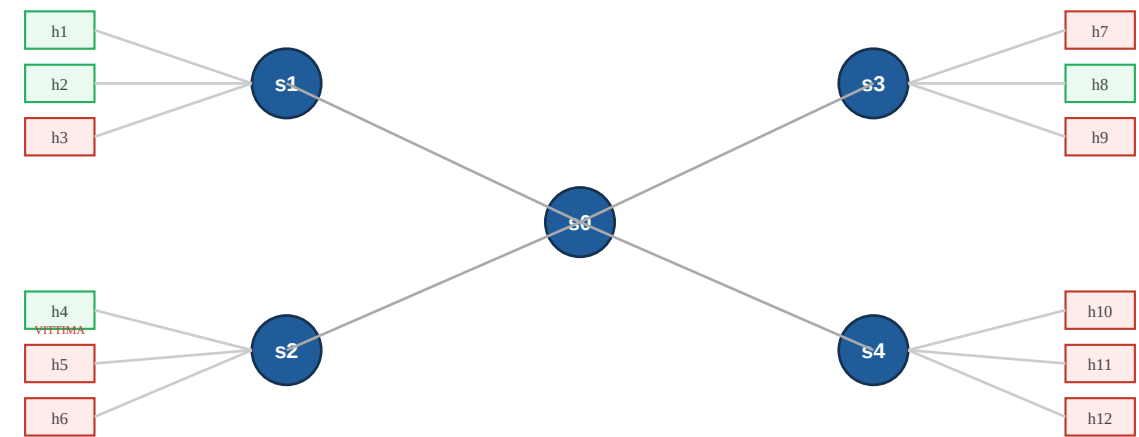


Figura 1 - Topologia core/access. Gli attaccanti (in rosso) sono distribuiti su switch diversi; il traffico attraversa il core s0 e converge su h4.

Obiettivo e contributo

Punto	Descrizione
Problema affrontato	Rilevare traffico DoS/DDoS distribuito che degrada la comunicazione verso un host vittima.
Approccio	SDN con Ryu e OpenFlow 1.3: monitoraggio delle flow statistics, classificazione ML, mitigazione tramite regole DROP.
Miglioramento principale	Il blocco non interessa l'intera porta dello switch: viene installata una regola selettiva src_ip -> 10.0.0.4.
Scala della demo	Topologia core/access con 5 switch, 12 host e attaccanti distribuiti su switch differenti.

1. Ambiente, struttura ed esecuzione

Il progetto e' pensato per Ubuntu 22.04 con Mininet, Open vSwitch, Ryu, Python 3 e scikit-learn. La consegna contiene il codice della topologia, il controller Ryu, gli script di traffico, il dataset, il modello addestrato e i risultati sperimentali. Il controller Ryu viene eseguito da una copia locale presente nella cartella `ryu/`, indicando il percorso tramite la variabile `PYTHONPATH`.

Struttura essenziale della repository

```
DDOS/
  topology/network_core_access.py  # topologia Mininet core/access (demo)
  topology/auto_collect.py        # topologia automatica per la raccolta dataset
  controller/ml_detector_controller_final.py  # controller finale ML + mitigazione
  controller/dataset_collector_auto.py  # collector automatico del dataset
  scripts/udp_realistic_sender.py  # generatore UDP con rate/burst variabili
  scripts/traffic_udp_realistic_v8.mn  # scenario demo finale
  dataset/dataset_v8_training.csv  # dataset raccolto dalla rete
  ml/best_model.joblib  # modello Random Forest finale
  ml/model_metadata.json  # feature, soglia e metriche
  train_model.py  run_collect.sh  results/
```

Avvio del controller Ryu in locale

```
PYTHONPATH=./ryu python3 ryu/bin/ryu-manager <file_controller>
```

Raccolta del dataset e addestramento

La raccolta e' automatizzata: lo script avvia il collector, esegue le fasi di traffico (normale, attacco distribuito, attacco singolo) su piu' round e addestra il modello.

```
chmod +x run_collect.sh
./run_collect.sh  # raccolta automatica + training
python3 train_model.py  # riaddestramento senza nuova raccolta
```

Esecuzione della demo (due terminali)

```
# Preparazione: copiare il generatore dove gli scenari lo attendono
cp scripts/udp_realistic_sender.py /tmp/

# Terminale 1 - controller
PYTHONPATH=./ryu python3 ryu/bin/ryu-manager controller/ml_detector_controller_final.py

# Terminale 2 - rete Mininet
sudo python3 topology/network_core_access.py
# dentro Mininet:
source scripts/traffic_udp_realistic_v8.mn
```

Nota per la valutazione: il docente puo' valutare sia la demo con il modello gia' incluso, sia la riproducibilita' completa tramite raccolta dataset, addestramento e controller di detection.

2. Topologia, traffico e dataset

Topologia core/access

La rete usa uno switch centrale s0 e quattro switch di accesso s1-s4; ogni switch di accesso collega tre host. La vittima h4 e' connessa a s2, mentre le sorgenti di attacco sono distribuite su switch diversi: il traffico attraversa quindi piu' percorsi e converge sul server protetto. Tutti i link sono configurati con TCLink, banda 10 Mbit/s e ritardo 5 ms, per rendere osservabile la congestione.

Scenari di traffico

Scenario	Host e parametri
Normale (label 0)	h1, h2, h8, h5 generano UDP verso h4 con 2-25 pps, payload 80-300 byte e burst sporadici.
Attacco (label 1)	h3, h5, h6, h7, h9, h10, h11, h12 generano UDP verso h4 con 35-190 pps, payload 250-950 byte e burst piu' frequenti.
Demo finale	Traffico legittimo concorrente e attacco distribuito insieme: il controller deve distinguere le sorgenti.

Il generatore `udp_realistic_sender.py` introduce variabilita' reale: a ogni iterazione sceglie un rate casuale, una dimensione di payload casuale e, con una certa probabilita', un breve burst. Questo supera il limite dei flood costanti, troppo facili da distinguere.

Dataset utilizzato

Il dataset finale contiene 7051 righe, di cui 2001 normali e 5050 di attacco, ottenute da piu' round di raccolta. Le righe derivano da statistiche OpenFlow convertite in feature per flusso e in feature aggregate verso la destinazione h4.

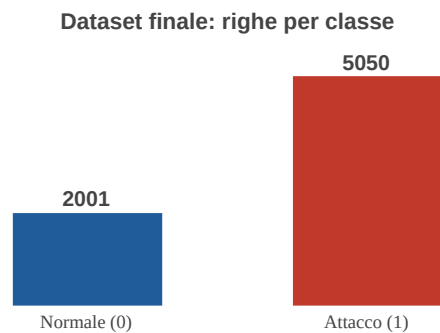


Figura 2 - Composizione del dataset finale per classe.

Pipeline di detection e mitigazione



Figura 3 - Dalla generazione del traffico alla mitigazione: Mininet, Ryu, estrazione feature, classificazione Random Forest, installazione della regola DROP.

3. Detection ML e mitigazione OpenFlow

Feature del modello

Il modello non usa le porte sorgente/destinazione: in un attacco reale le porte possono variare e produrrebbero regole troppo specifiche, oltre a generare un eccessivo numero di richieste al controller. Sono inoltre esclusi i contatori cumulativi (packet_count, byte_count, duration_sec), che dipendono dalla durata di osservazione e non dalla natura del traffico. La classificazione si basa su rate istantanei, dimensione media del pacchetto e contesto aggregato verso la vittima.

```
ip_proto, delta_packets, delta_bytes, packet_rate, byte_rate, avg_packet_size, dst_unique_src_count,
dst_total_packet_rate, dst_total_byte_rate, dst_active_flow_count, is_tcp, is_udp, flow_to_dst_ratio,
pps_per_source
```

Logica di decisione

Fase	Descrizione
Monitoraggio	Il controller invia OFPFlowStatsRequest agli switch; i contatori cumulativi vengono trasformati in delta e rate istantanei.
Classificazione	Random Forest con soglia 0.35; il modello stima la probabilità di attacco per ogni flusso attivo.
Conferma	Una sorgente viene bloccata solo dopo 4 rilevazioni consecutive sospette, riducendo i falsi positivi momentanei.
Mitigazione	Regola OpenFlow priority=200, hard_timeout=300s, match su ipv4_src e ipv4_dst=10.0.0.4, actions=drop.

Scelte progettuali rilevanti

Limite evitato	Soluzione adottata
Over-blocking	Si blocca solo la coppia sorgente sospetta -> vittima, non l'intera porta dello switch.
Soglia statica	Le condizioni aggregate restano contesto di log; la decisione finale è guidata dal modello ML.
Saturazione del controller	Il forwarding non dipende dalle porte L4: si evita una entry per ogni porta casuale e il sovraccarico del control plane.
Attacco distribuito	Attaccanti su switch diversi; le feature includono numero di sorgenti, rate totale e flussi attivi verso h4.
Blocco permanente	La regola DROP ha hard_timeout=300s: la rete recupera senza intervento manuale.

Esempio di regola installata (dum reale)

```
priority=200, ip, nw_src=10.0.0.3, nw_dst=10.0.0.4 actions=drop
priority=200, ip, nw_src=10.0.0.5, nw_dst=10.0.0.4 actions=drop
priority=200, ip, nw_src=10.0.0.11, nw_dst=10.0.0.4 actions=drop
```

4. Valutazione, risultati e checklist

Metriche del modello finale

Soglia	Accuracy	Precision	Recall	F1	F1 baseline (if)
0.35	0.752	0.743	0.999	0.853	0.158

La soglia 0.35 privilegia il recall sugli attacchi mantenendo una precisione accettabile: in una mitigazione DDoS e' preferibile non lasciar passare traffico malevolo persistente, dato che la conferma su rilevazioni consecutive filtra comunque i falsi allarmi momentanei. Il confronto con una regola a soglia singola (`if packet_rate >= k`) e' significativo: il baseline raggiunge F1 0.158, il modello 0.853. Le feature piu' rilevanti risultano il rate aggregato verso la destinazione e i rapporti `pps_per_source` e `flow_to_dst_ratio`, coerenti con la natura distribuita dell'attacco.

Evidenza sperimentale dalla demo

Al termine della demo sono stati salvati i dump delle flow table. La tabella riporta il numero di regole DROP installate per switch verso la vittima h4.

Switch	Regole DROP verso 10.0.0.4
s0 (core)	7
s1	2
s2 (vittima)	8
s3	3
s4	3

Cosa osservare durante la demo

Verifica	Risultato atteso
Avvio controller	Caricamento di modello, soglia e lista feature senza errori.
Avvio Mininet	5 switch, 12 host e controller remoto connesso.
Esecuzione traffico	Log con probabilita' ML, pps/bps e contesto aggregato verso 10.0.0.4.
Mitigazione	Messaggi di blocco dopo rilevazioni consecutive sospette.
Dump OpenFlow	Regole DROP priority=200 sulle sorgenti sospette, non blocco totale della porta.

Limiti dichiarati e sviluppi futuri

Il sistema e' stato validato in emulazione su traffico UDP realistico con burst e rate variabile. Un limite noto e' che, durante una congestione, host legittimi molto attivi possono occasionalmente essere classificati come sospetti; la conferma su rilevazioni consecutive ne riduce l'impatto. Estensioni naturali: addestramento con traffico TCP e ICMP piu' ampio, validazione su topologie con percorsi multipli, soglia adattiva e una blocklist/whitelist esterna modificabile senza riavviare il controller.

Checklist di consegna: inserire nomi, matricole e link al repository GitHub; caricare codice e PDF (massimo 5 pagine); non includere cartelle di librerie esterne non necessarie; verificare che `model_metadata.json` e `best_model.joblib` siano coerenti.